

بسم الله الرحمن الرحيم

گزارش پیاده سازی مقاله با عنوان:

شناسایی فرصت های بازنویسی متدهای استخراج شده حاصل از تجزیه متدها

ارائه کننده: محمد رحیمی

نیم سال دوم تحصیلی - خرداد ۹۹

فهرست مطالب

۴	۱. چکیده
۴	۲. مقدمه
۴	۲.۱ شرح مسئله
۵	۲.۲ راهکارهای مسئله
۵	۲.۳ داده‌ها و نتایج
۶	۳. مفاهیم اولیه
۶	۳.۱ گراف جریان کنترلی
۶	۳.۲ مجموعه reachable
۶	۳.۳ مجموعه Dominant
۶	۳.۴ مجموعه Post Dominant
۶	۳.۵ گراف وابستگی کنترلی
۶	۳.۶ مجموعه Dom
۶	۳.۷ مجموعه boundary blocks
۷	۳.۸ مجموعه Gen
۷	۳.۹ مجموعه In
۷	۳.۱۰ مجموعه kill
۷	۳.۱۱ مجموعه Out
۷	۴. پیاده‌سازی‌ها
۷	۴.۱ گراف جریان کنترلی
۸	۴.۲ بدست آوردن مجموعه Dominant
۸	۴.۳ مجموعه‌ی Reach
۹	۴.۴ مجموعه‌ی Post-Dominant
۱۰	۴.۵ گراف وابستگی جریان
۱۰	۴.۶ مجموعه‌ی Dom
۱۱	۴.۷ مجموعه‌ی Blocks
۱۱	۴.۸ مجموعه‌ی Gen
۱۲	۴.۹ مجموعه‌ی Kill
۱۲	۴.۱۰ مجموعه‌های In و Out
۱۳	۵. ارزیابی و مدت زمان اجرای توابع نوشته شده در این برنامه
۱۴	۵.۱ ایده جدید

۱. چکیده

استخراج قطعات کد به صورت متدهای جداگانه یکی از پرکاربردترین فعالیت‌های refactoring است به‌طوری‌که این امکان را برای تجزیه و تحلیل متدهای بزرگ و پیچیده فراهم می‌کند و می‌تواند با سایر روش‌هایی که کد را دچار تغییر می‌کند ترکیب شود تا بسیاری از مشکلات طراحی را رفع کند.

با وجود آنکه استخراج متد در Refactor کردن کد، کیفیت کد را بهبود می‌دهد، محدودیت‌هایی وجود دارد که شناسایی یک قطعه کد را به عنوان یک تابع مجزا پشتیبانی کند به‌طوری‌که کد استخراج شده یک تابع جدید باشد.

هدف از این رویکرد استخراج متدهای پالایش شده به صورت اتوماتیک است به‌طوری‌که باید یک برش و حالت‌هایی که باعث تغییر حالت یک آبجکت دریافت شده می‌شود محاسبه گردد. (Object state slice)

علاوه بر این، مجموعه‌ای از قوانین در مورد حفظ وابستگی‌های موجود پیشنهاد شده است که مانع از ریفکتور کردن برش‌هایی می‌شود که استخراج آنها احتمالاً می‌تواند باعث تغییر در رفتار برنامه شود.

این رویکرد توانایی گرفتن اسلایس‌ها از یک کد پیاده‌سازی شده به عنوان یک تابع جداگانه، حل شکاف‌های طراحی موجود و تاثیراتش بر روی چسبندگی‌های تجزیه و استخراج متد و توانایی‌اش در دوباره ارائه کردن رفتار برنامه مورد ارزیابی قرار می‌دهد.

به علاوه Precision و Recall محاسبه شده و به صورت مستقل در نرم‌افزاری که به عنوان یک مجموعه اهداف طلایی توسعه داده شدند مورد ارزیابی قرار گرفتند. (Nikolaos Tsantalis 2011)

۲. مقدمه

طبق چندین مطالعه تجربی پروسیجر / ماژول‌ها با اندازه بزرگ، پیچیدگی بالا و انسجام کم به زمان و تلاش زیاد برای فهمیدن، اشکال زدایی، تست و دستکاری نیاز دارند. (Nikolaos Tsantalis 2011)

۲.۱ شرح مسئله

برای اولین بار برای حل این نوع از مسائل و مشکلات راه حلی توسط فاولر و همکارانش در سال ۱۹۹۹ تحت عنوان Refactoring extract Method مطرح شد که کد را با شکستن متدهای بزرگتر به کوچکتر ساده می‌کند. به‌طوری‌که این متدهای کوچک قابلیت استفاده مجدد را دارند.

با این حال در حال حاضر، رویکردهای تحقیقاتی به این موضوع توجه می‌کنند که به صورت اتوماتیک Statementsهایی را استخراج کرده و بدون پشتیبانی و به صورت اتوماتیک این جملات را به متدهای کوچک تبدیل کنند.

استخراج متدهای ریفکتور شده برای برطرف کردن عیوب طراحی از قبیل

۱- Duplicated code که ساختار موجودی کد در بیش از یک مکان که به صورت یک تک متد استخراج شده یکسان باشد.

۲- Long Method: در موقعیت‌هایی که یک بخش از متدهای بزرگ و پیچیده به عنوان یک فانکشن مجزا در یک متد جدید تعریف شود.

۳- Feature Envy: در موقعیت‌هایی اتفاق می‌افتد که با استفاد از چند داده از کلاس‌های دیگر مقداردی اولیه می‌شوند

۴- به علاوه در تبدیلات Extract Method Refactoring بستر مهاجرت از برنامه Object-Oriented به Oriented Paradigm فراهم می‌گردد. (Nikolaos Tsantalis 2011)

۲.۲ راهکارهای مسئله

رویکرد مورد نظر برای ایجاد متدهای کوچکتر محاسبه‌ی اسلایس است. در این مقاله بر حسب فیلدهای یک کلاس و متغیرهای یک تابع عملیات اسلایسینگ انجام می‌شود. به طور کلی دو ریکرد کلی در این مسئله برای استخراج متد وجود دارد که عبارتند از:

۱- در دسته‌ی اول متغیرهایی که مقدارشان به وسیله‌ی جملات نسبت داده شده در سرتاسر بدنه متد اصلی اصلاح می‌شود.

۲- دسته دوم به آبجکت رفرنس‌هایی اطلاق می‌شود (که متغیر محلی هستند یا فیلدهایی از یک کلاس که تحت تاثیر متد اصلی می‌باشد) که به وسیله‌ی یک متد فراخوانی شده در بدنه‌ی اصلی تحت تاثیر قرار می‌گیرد.

در مورد اول هدف استخراج محاسباتی است که یک متغیر را تحت تاثیر قرار می‌دهد به عبارتی در این مرحله باید یک اسلایس محاسباتی کامل بدست آید. در حالیکه در مورد دوم هدف استخراج جملات اصلاح شده از یک آبجکت درون اسکوپ‌هایی از بدنه‌ی اصلی است. (Nikolaos Tsantalis 2011)

۲.۳ داده‌ها و نتایج

متدهای مورد استفاده شده در این پیاده‌سازی متدی از کلاس Customer به نام Statement و متد دیگری از کلاس Code Example به نام Test می‌باشد که به پیوست این گزارش وجود دارد و درون فایل txt ذخیره گردیده و قابل استفاده می‌باشد.

در این پروژه بخشی از مقاله‌ی "Identification of extract method refactoring opportunities for the decomposition of methods" پیاده‌سازی گردیده است. پیاده‌سازی‌ها با استفاده از زبان سی‌شارپ و کامپایلر این زبان یعنی Roslyn انجام گرفت. کارهای انجام گرفته در این پروژه عبارتند از:

- ۱- بدست آوردن گراف جریان کنترلی (CFG)
- ۲- محاسبه مجموعه Reach (ابتکاری)
- ۳- بدست آوردن معکوس گراف جریان کنترلی (CFGR) (Parsa 2005)
- ۴- محاسبه مجموعه Dominant (Parsa 2005)
- ۵- محاسبه مجموعه Post Dominant (Parsa 2005)
- ۶- محاسبه‌ی گراف وابستگی جریان (CDG) (Parsa 2005)
- ۷- محاسبه مجموعه Dom (Maruyama 2001)
- ۸- محاسبه مجموعه Blocks (Maruyama 2001)
- ۹- محاسبه‌ی مجموعه‌های Gen , Kill (Parsa 2005)
- ۱۰- محاسبه مجموعه‌ی In , Out با استفاده از الگوریتم موجود در جزوه دکتر سعید پارسا (Parsa 2005)

پیاده‌سازی‌های انجام شده توانستند به خوبی مجموعه‌های ذکر شده را بر روی هر تابع شناسایی و محاسبه نمایند.

۳. مفاهیم اولیه

به طور کلی مفاهیم اولیه این پروژه به شرح زیر عبارتند از:

۳.۱ گراف جریان کنترلی

گراف کنترل جریان از یک مجموعه Basic Block تشکیل شده است. هر Basic Block دنباله‌ای از دستورات متوالی است به طوریکه جریان کنترل از ابتدا تا انتهای آن بدون پریدن به شاخه‌ی دیگر ادامه دارد. همچنین به اولین نود هر بلاک، leader node گفته می‌شود.

۳.۲ مجموعه reachable

Reach (B_n) مجموعه‌ای از بلاک‌هاست که از طریق گراف کنترل جریان بدون در نظر گرفتن حلقه‌ها محاسبه می‌شود. مجموعه بلاک‌های قابل دسترسی از طریق Basic Block (B_n) جزء مجموعه Reach، بلاک B_n می‌باشند.

۳.۳ Dominant مجموعه

جهت یافتن لبه‌های برگشتی باید، لبه‌هایی را مشخص کرد که انتهای لبه بر ابتدای آن مسلط باشد. پس از افزودن گره‌های ابتدایی Start و انتهایی Stop به گراف جریان کنترلی، می‌توان مجموعه تحت تسلط را تعریف نمود. گره N_1 مسلط بر N_2 است اگر هر مسیر از Start به N_2 حتماً از N_1 عبور نماید.

۳.۴ Post Dominant مجموعه

برطبق تعریف گره Y بر Z پس تسلط دارد (البته در داخل گراف جریان) اگر و تنها اگر برای اینکه از گره Z به گره stop در داخل گراف جریان کنترل برسیم مجبور باشیم که حتماً از گره Y عبور کنیم. توجه داشته باشید که شاخه شاخه شدن اجرای برنامه‌ها در داخل گرافی به نام گراف جریان کنترل مشخص می‌شود.

۳.۵ گراف وابستگی کنترلی

گراف وابستگی کنترلی یک گراف جهت دار است. گره‌ها، جملاتی در قالب کد میانی بوده و لبه‌ها شاخص وابستگی کنترلی می‌باشند. بر روی هر چسب لبه‌ها جهت مشخص می‌شود. با استفاده از درخت تحت تسلط و گراف جریان کنترلی می‌توان گراف وابستگی کنترلی را ایجاد نمود. در واقع این عمل به نوعی، یک مرتب‌سازی گراف است. بدین منظور برای هر لبه (X, Y) باید نزدیکترین پدر مشترک دو گره X در درخت تحت تسلط مشخص شود. چنانچه این گره را L بنامیم L نمی‌تواند Y باشد. L می‌تواند X یا پدر X در درخت تحت تسلط باشد.

۳.۶ Dom مجموعه

به فرض آنکه نود r نودی است که به لیندر نود بیسیک بلاک B مسلط است $Dom(B)$ مجموعه‌ای از بلاک‌هاست که نود r به آن‌ها مسلط است. مسلط بودن به این معناست که در گراف وابستگی جریان یک ارتباط یا یک یال بین این نود و بیسیک بلاک وابسته وجود داشته باشد. نکته‌ی دیگر این است که این تسلط می‌تواند با واسطه نیز انجام گیرد.

۳.۷ boundary blocks مجموعه

برای محاسبه‌ی مجموعه‌ی blocks برای یک جمله کفایست ابتدا یک حلقه Foreach گذاشته و تمام نودها مورد بررسی قرار بگیرند. در این بررسی برای هر بلاک حلقه‌ی Foreach، $Reach(B)$ و $Dom(B)$ محاسبه می‌گردد. چنانچه بیسیک بلاک حاوی جمله‌ی مورد نظر، در این دو مجموعه وجود داشته باشد آن بلاک (یعنی بلاک B) به مجموعه‌ی Blocks آن جمله اضافه می‌گردد.

۳.۸ مجموعه Gen

مجموعه تعاریفی که در داخل یک بلاک اولیه وجود دارد و با تعریف مجدد مواجه نشده باشد.

۳.۹ مجموعه In

مجموعه تعاریفی که از خارج از بلاک اولیه به آن بلاک اولیه می‌رسند.

$$In[B] = \cup Out(p) \forall p \in Predessessore(B)$$

۳.۱۰ مجموعه kill

مجموعه تعاریفی که به بلاک اولیه رسیده‌اند و بواسطه تعریف مجدد در داخل بلاک اولیه از میان رفته‌اند.

$$Kill[B] = In[B] - Gen[B]$$

۳.۱۱ مجموعه Out

مجموعه تعاریفی که از داخل یک بلاک اولیه خارج می‌شوند و به بلاک‌های بعدی می‌رسند.

$$Out[B] = Gen[B] + (In[B] - kill[B])$$

۴. پیاده‌سازی‌ها

در این بخش به تشریح الگوریتم‌های ایجاد شده جهت پیاده‌سازی گراف جریان کنترلی ، مجموعه تسلط ، پس تسلط ، گراف وابستگی جریان، مجموعه‌ی Dom و Blocks و پیاده‌سازی مجموعه‌های In ، Out ، Gen و Kill پرداخته می‌شود.

۴.۱ گراف جریان کنترلی

کامپایلر رازلین دارای کتابخانه‌ای به نام `Microsoft.CodeAnalysis.FlowAnalysis.ControlFlowGraph` است که گراف وابستگی جریان توسط این کتابخانه ارائه می‌گردد. برای بدست آوردن گراف وابستگی جریان دانستن چند نکته الزامی است:

- ۱- بر خلاف معمول برای Parse کردن کد علاوه بر خود کد نیاز به یک متغیر به نام Options از نوع `CSharpParseOptions` می‌باشد. این کلاس حاوی ویژگی‌ها و پیشنهادهایی است که تحلیل جریان را نشان می‌دهد به عبارتی برای کامپایل کردن کد و بدست آوردن گراف جریان کنترلی وجود متغیر Options ضروری است.
- ۲- کامپایلر رازلین برای ایجاد گراف جریان کنترل احتیاج به دو ورودی دارد. ورودی اول بدنه‌ی متد و ورودی دوم یک متغیر از نوع model می‌باشد که از کامپایل کردن tree توسط متد `GetsemanticModel` از کلاس `Compilation` بدست می‌آید.
- ۳- وظیفه کامپایل کردن و ایجاد گراف جریان کنترلی به عهده‌ی تابعی به نام `MControlFlowGraphCompilation` نهاده شده است. ورودی این تابع رشته کد مورد پردازش و خروجی آن گراف جریان کنترلی می‌باشد.
- ۴- برحسب نیاز و با توجه به اینکه در مراحل آتی نیازمند بدست آوردن فرزندان و اجداد هر نود می‌باشد تابعی به نام `MFindPredecessore` نوشته شد که اجداد هر نود را محاسبه می‌کند.

با توجه به محدودیت‌های امکاناتی موجود در گراف ارائه شده توسط کامپایلر رازلین گراف جدیدی ارائه گردید که هر بلاک آن دارای خصوصیت `Id` ، `ParentList` ، `ChildList` ، `basicBlock` ، `dataFlowAnalysis` و `Gen.Statements` ، `Kill` ، `In` ، `Out` می‌باشد. این گراف توسط متد `MControlFlowGraph` بدست می‌آید.

۴.۲ بدست آوردن مجموعه Dominant

مجموعه Dominant از روی گراف جریان کنترلی محاسبه می‌گردد. این مجموعه برای محاسبه‌ی گراف وابستگی جریان استفاده می‌شود. با استفاده از الگوریتم زیرمجموعه‌ی تسلط برای هر نود بدست می‌آید. (Parsa 2005)

Algorithm Dominant

1. $Dominant(Start) = \{Start\}$
2. forall node $\neq$ Start in ControlFlowGraph do
 $Dominant(node) = \text{All Nodes}$
endfor
3. Changes = true
4. while Changes do
5. Changes = false
6. forall node $\neq$ Start in ControlFlowGraph do
7. $NewDominant = \{node\} \cup \{ \cap Dominant(p), \forall p \in \{predecessors(node)\} \}$
8. if $Dominant(node) \neq NewDominant$
9. then changes = true
10. endif
10. $Dominant(node) = NewDominant$
 endfor
- endwhile

تنها نکته‌ای که در کد بالا وجود دارد در خط ۷ می‌باشد. به اینصورت که برای پیاده‌سازی این خط از این منطق استفاده شده است که چنانچه نود تنها یک پدر داشته باشد مجموعه Dominant آن به مجموعه Dominant نود اضافه می‌گردد. در غیر اینصورت (یعنی نود بیش از یک پدر داشته باشد) بین مجموعه Dominant های والدین نود اشتراک گرفته می‌شود و حاصل اشتراک به مجموعه Dominant این نود اضافه می‌گردد.

۴.۳ مجموعه Reach

مجموعه Reach از روی یک گراف جریان کنترلی بدون یال بازگشتی محاسبه می‌گردد. بنابراین در گام اول لازم است یال‌های بازگشتی از گراف جریان کنترلی حذف شوند. بدین منظور از یک لیست استفاده می‌گردد. به اینصورت که هر بار نودی که پیمایش شود این نود درون یک لیست قرار می‌گیرد و نود جدید بررسی می‌گردد. بررسی به اینگونه است که فرزندان نود جدید نباید در لیست نودهای پیمایش شده قرار داشته باشند. چنانچه فرزندان در لیست نودهای پیمایش شده قرار داشته باشند به یک یال بازگشتی برخورد کردیم و نودی که باعث ایجاد یال بازگشتی شده است باید از لیست فرزندان نود مورد بررسی حذف گردد. شبه کد این الگوریتم در زیر قابل مشاهده است:

Algorithm check edge back

```
1. List<node> travelsNode = new List<node>();
2. Foreach(node in tree ){
3. Foreach(item in travelsNode){
4. If (node.childs.exist(item))
5. Node.childs.delete(item);
6. travelsNode.Add(node);
}
}
```

پس از آنکه یال‌های بازگشتی از گراف جریان کنترلی حذف شدند نوبت به بدست آوردن مجموعه‌ی Reach برای هر نود می‌رسد. برای این کار از یک صف استفاده می‌شود. فرض کنید می‌خواهیم مجموعه Reach را برای نودی مانند A بدست بیاوریم. ابتدا نود A درون صف قرار می‌گیرد و سپس از درون صف حذف شده و به مجموعه‌ی Reach آن نود اضافه می‌گردد. سپس فرزندان نود A درون صف قرار می‌گیرند و اینکار آنقدر ادامه پیدا می‌کند تا صف خالی شود. به اینصورت مجموعه‌ی Reach برای یک نود بدست می‌آید. شبه کد زیر نشان دهنده‌ی عملیات بالا می‌باشد:

Algorithm Reach

```
1. Queue() queue = new Queue();
2. Foreach(node in tree);
3. queue.enqueue(node);
4. List<Node> reaches;
5. While(queue.count > 0) {
6.   Node reachNode = queue.dequeue();
7.   reaches.add( reachNode);
8.   queue.enqueue(reachNode.childs)
}
}
```

۴.۴ مجموعه‌ی Post-Dominant

برای بدست آوردن مجموعه‌ی Dominant لازم است گراف جریان کنترلی معکوس گردد (Parsa 2005). در گام اول برای معکوس کردن گراف جریان کنترلی کافیسیت لبه‌ها برعکس شوند. برای این منظور برای هر Basic block کافیسیت فرزندان هر نود در لیست والد آن نود و جد نود در لیست فرزند قرار گیرد. بدین صورت گراف جریان کنترلی معکوس می‌گردد.

Algorithm ReverseCFG

```
1. List<node> temp = new List<temp> ();
2. Foreach(node in CFG) {
3. temp.add(node.childs);
4. node.childs.clear();
5. node.childs.add(node.parents);
```

```

6. node.parents.clear();
7. node.parent.add(temp);
8. temp.clear();
}

```

در گام بعد لازم است مجموعه PostDominant بر طبق شبه کد زیر از روی گراف Reverse CFG برای هر نود محاسبه گردد:
(Parsa 2005)

```

1.D(n0) = {n0 }
2. for each node n in N- {n0} do
3.     D(n) = N
    endfor
4. while changes to any D(n) occur do
5.     for n in N- {n0} do
6.         D(n)={node} ∪ { ∩ Dominant(p), ∀ p ∈ {predecessors(node)}}
    endfor
endwhile

```

۴.۵ گراف وابستگی جریان

الگوریتم مورد استفاده برای گراف وابستگی جریان به شرح زیر است: (Parsa 2005)

- ۱- گرهی Start را به لیست گراف وابستگی جریان اضافه نمایید.
 - ۲- درخت پس تسلط را ایجاد کن.
 - ۳- وابستگی‌های کنترلی را طبق مراحل زیر تکرار کن.
- برای ایجاد لبه‌ها فرض کنید نودی مانند A در نظر گرفته می‌شود. سپس بررسی می‌گردد که چنانچه فرزندان مستقیم آن در لیست Post Dominant(A) نباشند این نود به همراه فرزندان آن به عنوان یک لبه در نظر گرفته می‌شود. این کار برای تمام نودهای موجود در **گراف جریان کنترلی** چک می‌گردد.
 - نزدیکترین جد مشترک (A,B) مورد جستجو قرار می‌گیرد. این جد L نامیده می‌شود.
 - در درخت پس تسلط از گرهی B به سمت L در خلاف جهت پیمایش انجام می‌شود. همه‌ی نودهای مشاهده شده به جز L علامت زده می‌شوند.
 - تمام جملات علامت خورده وابستگی کنترلی به A هستند.

۴.۶ مجموعه‌ی Dom

مجموعه‌ی Dom برای هر بلاک (با توجه به Statement‌های آن) از روی گراف وابستگی جریان محاسبه می‌گردد. شبه کد زیر نحوه محاسبه‌ی این مجموعه را نشان می‌دهد.

Algorithm Dom:

```

1.List<Dom> ListDom = new List<Dom>();
2.Queue queue = new Queue();
3.Node node = CDG.nodes.Find(statement);
4.queue.enqueue(node.parents);
5.while(queue.count > 0){
6.  Node domNode = queue.Dequeue();
7.  ListDom.Add(domNode);
8.  queue.Enqueue(node.childs);
}

```

۴.۷ مجموعه‌ی Blocks

مجموعه‌ی Blocks طبق شبه کد زیر محاسبه می‌گردد.

```

List<Node> blocks = new List<Node>();
Block = CDG.FindBlock(Statement);
For(Block in CDG){
    Reach = Reach(Block);
    dom = doms(Block);
    if(reach.exist(block) && Dom.exit(block))
        Blocks.Add(Block);
}

```

۴.۸ مجموعه‌ی Gen

هر عنصر مجموعه‌ی Gen، به عنوان یک مجموعه دو عنصری مانند (x, i) در نظر گرفته می‌شود. به طوریکه x نام متغیر و i شماره جمله‌ای است که این متغیر در آن تعریف شده است.

در گام اول یک کلاس به نام Gen ساخته شده است که دارای فیلدهای variable از نوع string و id از نوع int است. در ادامه Property ها و Constructor ها و dispose های کلاس ساخته می‌شود.

پس از آن نوبت به محاسبه‌ی مجموعه‌ی Gen می‌رسد. برای این کار یک تابع به نام Gen ایجاد می‌گردد. منطق پیاده‌سازی مجموعه Gen نیز به این صورت است که در تمام بلوک‌های کد، جملاتی در نظر گرفته می‌شوند که انتسابی باشند $(x = 1)$ یا ماهیت انتسابی باشند $(x++)$ یا $(Read(i))$. سپس چنانچه این جمله آخرین جمله‌ای باشد که متغیری مانند x در آن مقداردهی شده باشد شماره جمله به عنوان شماره مجموعه (x,i) و متغیر آن به عنوان متغیر این مجموعه در نظر گرفته می‌شود.

```

Foreach (block in Basic blocks) {
    List<Gen> gens = new List<Gen> ();

```

```

Foreach (variable in block.variable){
    Gen() gen = new Gen();
    gen = block.Statements.Find(variable).last();
    gens.Add(gen);
}
}

```

۴.۹ مجموعه‌ی Kill

هر عنصر مجموعه‌ی kill به عنوان یک مجموعه دو عنصری مانند (x, i) در نظر گرفته می‌شود. به طوریکه i شماره خط و x نام متغیر مورد نظر می‌باشد. برای پیاده‌سازی این مجموعه کلاسی به نام kill در نظر گرفته می‌شود. این کلاس دارای دو فیلد به نام `variable` و `lineId` می‌باشد که به ترتیب نام متغیر و شماره خط می‌باشد. در ادامه `Property` ها و سازنده‌ها و مخرب کلاس ایجاد می‌گردد.

سپس نوبت به پیاده‌سازی تابعی می‌گردد که مجموعه‌ی kill را محاسبه نماید. برای اینکار ابتدا لازم است مجموعه‌ی In از رابطه‌ی زیر محاسبه گردد.

$$In[B] = \cup Out(p) \forall p \in Predessores(B)$$

پس از محاسبه‌ی مجموعه‌ی In[B] نوبت به محاسبه مجموعه‌ی Kill[B] می‌رسد. مجموعه‌ی kill[B] از رابطه‌ی زیر محاسبه می‌گردد.

$$Kill[B] = In[B] - Gen[B]$$

با توجه به روابط بالا شبه کد زیر این مجموعه را محاسبه می‌کند.

```

Foreach (block in basicBlocks) {
    List<BasicBlock> parentBlocks = Block.parent();
    Foreach (basicBlock in parentBlocks) {
        List<In> ins = new List<In> ();
        Ins.add(gens[basicBlock]);
    }
    Ins.remove(gens[block]);
}

```

۴.۱۰ مجموعه‌های In و Out

هر عنصر مجموعه‌ی In و Out به عنوان یک مجموعه دو عنصری مانند (x, i) در نظر گرفته می‌شود. به طوریکه i شماره خط و x نام متغیر مورد نظر می‌باشد. برای پیاده‌سازی این مجموعه کلاس‌های به نام In و Out در نظر گرفته می‌شود. این کلاس‌ها

دارای دو فیلد به نام variable و lineId می‌باشد که به ترتیب نام متغیر و شماره خط می‌باشد. در ادامه Property ها ، سازنده‌ها و مخرب کلاس ایجاد می‌گردد.

سپس نوبت به ایجاد متدی برای پیاده‌سازی مجموعه‌های In و Out می‌رسد. این متد با توجه به الگوریتم موجود در جزوه دکتر سعید پارسا که به شرح زیر است محاسبه می‌گردد.

Input: Gen and Kill sets for each basic block

Output: In and Out sets for each basic block

1.for each node n does

OUT[n] = Gen[n]

IN[n] = null

Endfor

2. change = true

3.while change do

4. change = false

5. for each node n in traverse (CFG) do

6. IN[n] = $\cup$ OUT[P], where p is an immediate predecessor of n

7. OLDOUT = OUT[n]

8. OUT[n] = Gen[n] $\cup$ (IN[n] – kill[N])

9. if OUT[n] $\neq$ OLDOUT

10. then change = true;

Endif

Endfor

Endwhile

۵. ارزیابی و مدت زمان اجرای توابع نوشته شده در این برنامه

مدت زمان اجرای توابع نوشته شده به شرح جدول زیر است:

Name	Time(ms)
Dominant	2400ms
Post-Dominant	2600ms
Reach	2500ms
Dependency Graph	3000ms
Dom	3100ms
Blocks	3300ms
Gen, Kill, In, Out	3700ms

۵.۱ ایده جدید

ایده‌ی من برای ادامه‌ی این مقاله تعویض اسلایس است. با توجه به اینکه این مقاله از رویکرد Block-Based Slicing استفاده نموده است و این رویکرد اولین بار توسط خانم ماریاما در سال ۲۰۰۱ ارائه گردیده است (Maruyama 2001) بهتر است رویکردهای جدید اسلایسینگ مورد آزمایش قرار بگیرد و کارایی آن‌ها ارزیابی گردد. از جمله روش‌های جدید اسلایسینگ می‌توان به مقاله با عنوان an Accurate Object Oriented slicing based on Sub-statement Level Dependence Graph اشاره نمود. این مقاله کد را به سه سطح تقسیم می‌شود. در سطح اول جملات بر حسب سطوح استفاده سطح‌بندی می‌گردند. در سطح دوم شاخص‌های جدیدی برای اسلایسینگ معرفی می‌گردد که نه تنها اشیاء و آبجکت‌ها را در نظر می‌گیرد بلکه آبجکت‌های چندریختی را نیز حمایت می‌کنند. در سطح سوم الگوریتم برش‌بندی به نام STS برای انجام فرایند برش طراحی گردیده است. (Wang Lulu 2020)

References

- Maruyama, Katsuhisa. 2001. "Refactoring, Automated Method-Extraction." *Association for Computing Machinery* 31–40. doi:<https://doi.org/10.1145/375212.375233>.
- Nikolaos Tsantalis, Alexander Chatzigeorgiou. 2011. "Identification of extract method refactoring opportunities for the decomposition of methods." *Jornal of system and software* 1757-1782. doi:<https://doi.org/10.1016/j.jss.2011.05.016>.
- Parsa, Saeid. 2005. "Advance Compiler." Booklet, Iran University of Science and Technology, Tehran. <http://www.dr-parsa.ir>.
- Wang Lulu, Li Bixin*, Kong Xianglong. 2020. "an Accurate Object Oriented slicing based on Sub-statement." *Journal Pre-proof* (Proceedings of the 14th International Working Conference on Variability Modelling of Software-Intensive Systems) 1-5. doi:<https://doi.org/10.1016/j.infsof.2020.106369>.